



# Generative AI in Web Applications

## Task

[Visit our website](#)

# Introduction

Over the previous lessons, you have explored many of the technologies that underpin modern AI systems. You began by examining the fundamentals of artificial intelligence and machine learning before exploring embeddings, transformers, and large language models (LLMs). You have also built AI-powered interfaces, integrated AI APIs into applications, and examined how conversational experiences can be incorporated into modern web development.

At this stage, it is important to shift focus from understanding how generative AI works to understanding what developers can build with it. While the underlying technologies are fascinating, their true impact lies in how they are changing the nature of software itself. For many years, web applications have largely been designed around managing information. Applications collect data, store it, retrieve it when needed, and present it to users. Whether you are working with an e-commerce platform, a learning management system, a project management tool, or a social media application, the core responsibility of the software is typically the same: managing existing information.

Generative AI introduces a new capability. Applications are no longer limited to presenting information that already exists. They can now create information dynamically in response to user needs. This shift is influencing how applications are designed, how features are developed, and how users interact with software.

In this lesson, you will explore how generative AI is changing modern web applications, examine the difference between traditional and generative systems, and investigate the new opportunities this technology creates for developers.

## From Information Systems to Generative Systems

To understand the impact of generative AI, it is useful to first consider how traditional web applications operate. Most applications are built around a relatively straightforward workflow. Information is created, stored within a database, and later retrieved when required. Users interact with this information through interfaces that allow them to search, browse, edit, or organize data.

Consider an online learning platform. Course authors create lessons, assessments, and learning resources. These materials are stored within a database and delivered to students when needed. Every learner typically receives the same content, regardless of their background knowledge, learning preferences, or previous experience.

Similarly, an e-commerce platform stores product descriptions, pricing information, and product images. Customers browse the catalogue and consume information that has already been created. These systems can be described as information systems because their primary purpose is to manage information. Generative AI introduces a different model. Rather than relying entirely on information that already exists, applications can generate new content when it is required. This means that content no longer needs to be written, designed, or prepared in advance for every possible situation.

Imagine returning to the learning platform example. Rather than presenting every learner with the same explanation of a concept, the platform could generate different explanations depending on a student's experience level. A beginner might receive a simplified explanation with everyday analogies, while a more experienced learner might receive a technical explanation supported by code examples. The application is no longer simply retrieving information. It is generating information.

This transition from information systems to generative systems represents one of the most significant shifts currently taking place in software development.

## Generative AI as an Application Capability

When many people think about generative AI, they immediately think about chatbots. While conversational interfaces are certainly one application of generative AI, they represent only a small portion of what the technology enables. A more useful way to think about generative AI is as a new application capability.

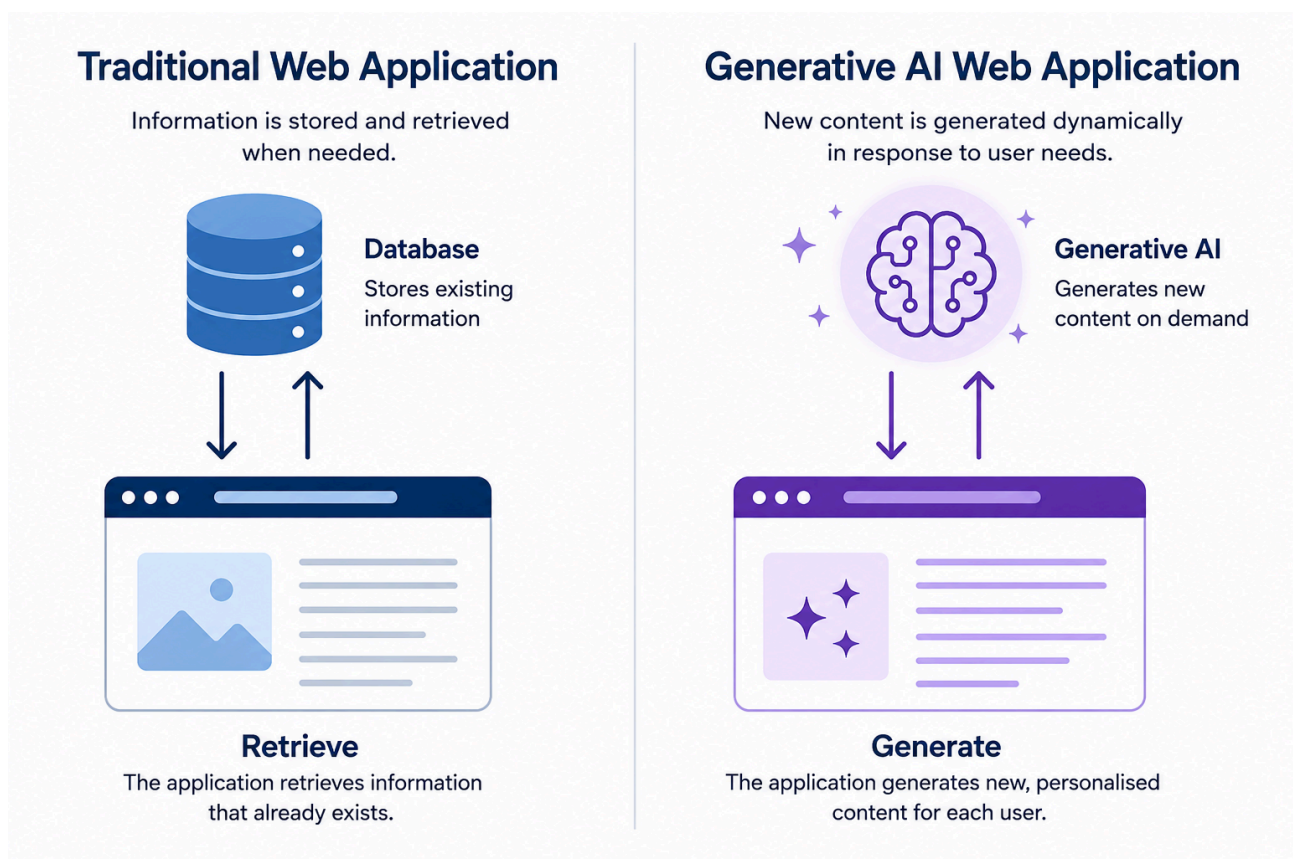
Throughout the history of web development, new capabilities have continuously been added to applications. Authentication allows applications to identify users. Payment systems allow applications to process transactions. Search functionality allows users to locate information quickly. Recommendation systems help users discover relevant content. Generative AI is another capability that developers can incorporate into software.

Rather than viewing AI as a standalone feature, developers can treat it as a service that enables applications to create content, explanations, recommendations, simulations, and personalized experiences. This perspective encourages developers to think more broadly about where AI can add value. For example, a project management application might use generative AI to create meeting agendas based on upcoming tasks. A learning platform could generate personalized revision exercises. A travel application could create customized itineraries tailored to individual interests.

In each case, the AI is not the application itself. It is a capability that expands what the application can do. As developers, this mindset helps us move beyond the idea of "adding a chatbot" and instead encourages us to think about how generation can be embedded throughout an application's functionality.

## Traditional AI Versus Generative AI

As generative AI becomes more common, it is important to understand how it differs from traditional AI systems. Traditional AI systems are typically designed to classify information, identify patterns, make predictions, or recommend actions. These systems analyse data and help applications make decisions. For example, a recommendation engine on an e-commerce website may analyse customer behaviour and suggest products that a user is likely to purchase. Similarly, a fraud detection system may analyse transactions and identify potentially suspicious activity.



These systems provide valuable insights, but they do not create new content. Generative AI introduces a new dimension. Rather than simply making a recommendation, a generative system can create content based on that recommendation. Imagine that an online store recommends three products to a customer. A traditional recommendation system would simply display the products. A generative AI system could go further by creating a personalized buying guide

explaining why each recommendation matches the customer's interests and previous purchases.

The difference is subtle but important. Traditional AI helps applications make decisions. Generative AI helps applications create content. Many modern applications combine both approaches. A recommendation engine may identify the most relevant products, while a generative AI system creates explanations, comparisons, and supporting content for those recommendations. Understanding this distinction helps developers identify where generative AI can provide value beyond prediction and analysis.

## Designing Generative Features

The introduction of generative AI changes how developers think about application features. Traditionally, most features are designed around predefined content and predefined workflows. Developers create a set of possible user journeys and build functionality around those journeys. Generative features operate differently.

Instead of selecting from existing content, users provide information that the application uses to create new content. The feature becomes less about retrieving information and more about generating outcomes.

Consider a fitness application. A traditional implementation might provide several predefined workout plans. Users browse the available plans and select the one that most closely matches their goals. A generative implementation takes a different approach. The user enters information such as their fitness level, available equipment, training goals, and available training time. The application then generates a personalized workout plan tailored to those requirements.

The workout plan does not exist until the user requests it. The application creates it dynamically. This same pattern can be applied across many different industries. A learning platform can generate personalized practice questions. A recruitment platform can generate interview preparation exercises. A productivity application can generate customized workflows. A travel platform can generate itineraries tailored to individual preferences.

As generative AI becomes more widely adopted, developers will increasingly need to think about how applications can create value through generation rather than simply through information management. One of the most important design questions developers can now ask is:

*"What content or experience could be generated dynamically instead of being predefined?"*

Answering that question often reveals opportunities for entirely new features and user experiences.



## Spot check 1

Let's see what you can remember from this section of the lesson.

1. What is the difference between an information system and a generative system?
  2. Why is it useful to think of generative AI as an application capability rather than simply a chatbot?
  3. How does generative AI differ from traditional AI systems?
  4. What is a generative feature?
  5. How might a learning platform use generative AI differently from a traditional learning platform?
- 

## Generative User Experiences

One of the most significant impacts of generative AI is its ability to transform user experiences. Historically, web applications have been designed around predefined content and predefined interactions. Developers create interfaces, workflows, and content that every user encounters in roughly the same way. Although personalization has existed for many years, it has typically been limited to recommending products, displaying relevant content, or adjusting interface settings. The underlying experience remains largely the same for all users.

Generative AI changes this model by allowing experiences themselves to become dynamic. Instead of delivering identical content to every user, applications can generate experiences tailored to individual needs, goals, preferences, and contexts. This means that two users interacting with the same application may receive entirely different outputs even when performing similar actions.

Consider an educational platform teaching JavaScript. A beginner who is learning arrays for the first time may require a simple explanation using real-world examples. An experienced developer may prefer a technical explanation that focuses on implementation details and performance considerations. Traditionally, supporting both learners would require multiple versions of the same lesson. A generative system can create these explanations dynamically, adapting the experience to the individual learner.

This idea extends beyond education. An e-commerce application could generate product comparisons based on a customer's interests. A travel platform could generate

different itineraries depending on whether a user prefers adventure activities, cultural experiences, or family-friendly attractions. A productivity application could generate personalized workflows that reflect a user's working habits.

The application is no longer simply delivering information. It is creating experiences. For developers, this represents a significant shift in user experience design. Instead of designing a single path through an application, developers increasingly design systems that can generate many possible paths.

## AI-Native Applications

As generative AI becomes more integrated into software development, a new category of software is emerging: AI-native applications. To understand this concept, it is useful to compare it with how AI has traditionally been incorporated into software. Historically, developers would build an application first and then add AI-powered features later. An online store might add an AI chatbot. A project management application might add AI-generated summaries. A learning platform might add an AI tutor.

In each case, AI acts as an enhancement to an existing product. AI-native applications take a different approach. In an AI-native application, the generative capability is the foundation of the product rather than an additional feature. The entire application is designed around what the AI can do. Consider modern AI coding assistants. The primary value of these tools is not their interface or navigation system. The value comes from the AI's ability to generate code, explain concepts, assist with debugging, and support development workflows. The surrounding application exists primarily to support that capability.

The same pattern can be seen in AI-powered research assistants, writing platforms, design tools, and content creation systems. These products are built around generations from the beginning. This trend is important because it is changing how software products are conceived. Increasingly, developers are asking not "How can we add AI to our application?" but rather "What kind of application becomes possible because AI exists?" This subtle shift often leads to entirely different product ideas and architectures.

## Generative AI and Developer Productivity

Generative AI is changing not only the applications developers build but also the way developers work. For many years, developers relied on documentation, tutorials, forums, and search engines to solve problems and learn new technologies. While these resources remain valuable, generative AI introduces a more interactive approach to

development. Modern AI systems can assist with generating code, explaining unfamiliar implementations, suggesting improvements, creating documentation, generating test cases, and helping developers understand complex systems.

The value of these tools is not that they eliminate the need for programming knowledge. Rather, they reduce the amount of time spent on repetitive tasks and allow developers to focus on higher-level problem solving.

For example, imagine that you are learning a new JavaScript framework. Instead of searching through multiple documentation pages to understand a concept, you can ask an AI assistant to explain the concept in the context of your existing knowledge. Similarly, if you need a starting point for a component, utility function, or API integration, an AI system can generate an initial implementation that you can refine and improve.

This changes the nature of software development. Developers spend less time producing routine code and more time evaluating solutions, designing systems, and making architectural decisions. As generative AI continues to improve, these productivity gains are likely to become an increasingly important part of professional development workflows.

## Vibe Coding and AI-Assisted Development

One of the more recent trends to emerge from generative AI is a development approach commonly referred to as *vibe coding*. Although the term is informal, it has become increasingly common within the software development community. Vibe coding describes a workflow where developers rely heavily on natural language instructions to generate software rather than manually implementing every component themselves. In a traditional workflow, a developer typically plans a solution, writes code, tests the implementation, and iterates until the desired outcome is achieved.

In a vibe coding workflow, the developer may begin by describing the desired functionality in plain language.

For example:

```
Build a responsive task management dashboard using React. Include task filtering, local storage persistence, dark mode support, and a dashboard summary showing completed and outstanding tasks.
```

A modern AI coding assistant may generate a substantial portion of the application automatically. The popularity of tools such as Cursor, GitHub Copilot, Windsurf, Replit AI,

Bolt, Lovable, and ChatGPT has accelerated this style of development. Developers can rapidly prototype ideas, experiment with unfamiliar technologies, and create working applications much faster than was previously possible.

However, it is important to understand both the benefits and limitations of this approach. Vibe coding can dramatically increase development speed, particularly during prototyping and early-stage development. It allows developers to explore ideas quickly and reduce the amount of repetitive coding required. At the same time, generated code is not automatically correct. AI-generated solutions may contain bugs, security vulnerabilities, performance issues, accessibility concerns, or architectural weaknesses that are not immediately obvious.

For this reason, strong programming fundamentals remain essential. Developers still need to understand how systems work, how to debug problems, how to test applications, and how to evaluate the quality of generated solutions. Generative AI can accelerate development, but responsibility for the final implementation remains with the developer.

## Code Generation as a Web Application Feature

When most developers think about AI-generated code, they think about developer tools. However, code generation can also become a feature within web applications themselves. Consider a platform designed to teach programming. Traditionally, instructors create examples, coding exercises, project ideas, and practice activities manually. While this approach works well, it limits the amount of personalized content that can be delivered to learners.

Generative AI allows applications to create programming content dynamically. A student learning JavaScript arrays might request examples related to online stores. Another student might prefer examples based on sports statistics. A third learner might want examples involving music playlists. Rather than storing hundreds of variations, the platform can generate examples on demand.

This creates opportunities for highly personalized learning experiences while reducing the amount of content that must be authored manually. The same concept can be extended to coding challenges, project briefs, assessment questions, interview preparation exercises, and debugging activities. As generative AI becomes more common, developers will increasingly view generation itself as a feature rather than merely a supporting technology.

# Generating Learning Activities

Imagine a learning platform that allows students to generate practice exercises based on a topic they are currently studying. A student selects a topic and the application sends a request to a backend service responsible for generating activities.

```
async function generateActivity(topic) {
  const response = await fetch("/api/generate-activity", {
    method: "POST",
    headers: {
      "Content-Type": "application/json"
    },
    body: JSON.stringify({
      topic: topic
    })
  });
  const data = await response.json();
  return data.activity;
}
```

The purpose of this code is not to demonstrate AI integration itself. You have already explored AI APIs earlier in the syllabus. Instead, the example illustrates a different way of thinking about application design. Traditionally, the application would retrieve a predefined exercise from a database. In a generative system, the activity may not exist until the user requests it. The application is generating content rather than retrieving it. This distinction is central to understanding how generative AI is changing web development.



## Spot check 2

Let's see what you can remember from this section of the lesson.

1. What is a generative user experience?
2. How does an AI-native application differ from an application that simply includes AI features?
3. How is generative AI changing software development workflows?
4. What is vibe coding?
5. Why does code generation still require developer oversight?

# Why Instructions Matter

As generative AI becomes increasingly integrated into web applications, developers are discovering that the quality of generated outputs is heavily influenced by the instructions provided to the AI system.

Traditional software behaves in a predictable manner. Given the same input, the application will typically produce the same output every time. Developers define the logic, write the conditions, and determine exactly how the system should respond.

Generative systems operate differently. While they still follow underlying rules and algorithms, the outputs they produce are influenced by the instructions they receive. Small changes in those instructions can produce significantly different results. Consider a learning platform that provides explanations of JavaScript concepts.

A simple instruction such as:

```
Explain JavaScript arrays.
```

may produce a useful explanation.

However, a more specific instruction such as:

```
Explain JavaScript arrays to a beginner using a real-world analogy  
and include a short code example.
```

is likely to produce a more targeted and useful response.

Both instructions are attempting to achieve the same objective, but the additional guidance helps the AI generate content that is better suited to the user's needs. This principle applies across a wide range of generative applications. Content generation systems, coding assistants, educational platforms, customer support tools, and recommendation engines all rely on instructions to guide the generation process.

For developers, this introduces a new design consideration. In addition to designing interfaces and application logic, developers must think carefully about how AI systems are instructed to perform tasks. The ability to create effective instructions has become an increasingly valuable skill because it directly influences the quality, relevance, and consistency of generated outputs.

In the next lesson, you will explore this topic in depth through prompt engineering. You will learn how developers structure instructions, provide context, define constraints, and guide AI systems toward producing more reliable and useful results.



## Spot check 3

Let's see what you can remember from the final section of this lesson.

1. Why do instructions play such an important role in generative AI systems?
2. How do generative systems differ from traditional software systems in the way they produce outputs?
3. Why might two different instructions produce different results even when using the same AI model?
4. How does prompt engineering relate to the development of generative applications?
5. Why is understanding instructions becoming an important skill for modern developers?



### Take note

The task below is **auto-graded**. An auto-graded task still counts towards your progression and graduation. Give it your best attempt and submit it when you are ready.

When you select “Request Review”, the task is automatically complete, you do not need to wait for it to be reviewed by a mentor.

You will then receive an email with a link to a model answer, as well as an overview of the approach taken to reach this answer.

Take some time to review and compare your work against the model answer. This exercise will help solidify your understanding and provide an opportunity for reflection on how to apply these concepts in future projects.

Once you've done that, feel free to progress to the next task.

---

# Instructions

In this activity, you will design and build a small web page that showcases a generative AI feature for a web application. The purpose of this task is to demonstrate your understanding of how generative AI can transform traditional applications into generative systems. Rather than writing a report, you will communicate your ideas through a professionally designed web page. Your submission must include:

- `index.html`
- `styles.css`
- `script.js`

Your web page should:

- Use semantic HTML.
- Apply professional styling using CSS.
- Be responsive on desktop and mobile devices.
- Include at least one interactive JavaScript element.
- Present information clearly and professionally.



## Auto-graded task

1. Choose one of the following application types:
  - a. Learning platform
  - b. E-commerce platform
  - c. Project management application
  - d. Fitness application
  - e. Travel planning platform
  - f. Job recruitment platform
2. Create a web page that explains and demonstrates your proposed generative AI feature. Your page should include the following sections.
  - a. **Section 1: Application Overview**

Introduce the application you selected and describe its primary users.  
Explain the purpose of the application and the problem it aims to solve.

b. **Section 2: Traditional Feature**

Identify a feature that would normally rely on predefined content.  
Examples include:

- Static product descriptions
- Fixed workout plans
- Prewritten study activities
- Standard travel itineraries
- Generic interview preparation guides

Explain how the feature would traditionally operate.

c. **Section 3: Generative AI Feature**

Redesign the feature using generative AI. Explain:

- What information the user would provide
- What content the application would generate
- How the generated output creates a more personalized experience
- Why the generative approach provides additional value

d. **Section 4: User Experience Walkthrough**

Demonstrate how a user would interact with the feature. You may use:

- Mock forms
- Example prompts
- Example generated outputs
- Interface wireframes created with HTML and CSS

Show what the user would enter and what the application would generate.

e. **Section 5: Developer Perspective**

Discuss the feature from a developer's perspective. Explain:

- Why generative AI is being used
- What benefits it provides
- What implementation challenges may exist
- Why developer oversight remains important

f. **Section 6: Instructions and Output Quality**

Provide two different instructions that could be used to generate content.  
For example:

Instruction 1

- Generate a workout plan.

Instruction 2

- Generate a beginner-friendly four-week workout plan for a 35-year-old office worker with access to only dumbbells and two training sessions per week.

Explain how the different instructions influence the generated result.

- g. While not mandatory for completing the assignment, you have the option to enhance your project by integrating an AI service to produce live, dynamic responses.

**Important:** Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.

---



## Spot check 1 answers

1. **What is the difference between an information system and a generative system?**

An information system primarily stores, retrieves, and displays existing information, while a generative system can create new content dynamically in response to user needs.

2. **Why is it useful to think of generative AI as an application capability rather than simply a chatbot?**

Because generative AI can be used throughout an application to create content, recommendations, explanations, and personalized experiences, not just conversational interactions.

3. **How does generative AI differ from traditional AI systems?**

Traditional AI systems typically focus on prediction, classification, and

recommendation, while generative AI creates new content such as text, code, images, or personalized outputs.

4. **What is a generative feature?**

A generative feature is a feature that creates content dynamically based on user input rather than relying entirely on predefined content.

5. **How might a learning platform use generative AI differently from a traditional learning platform?**

A generative learning platform can create personalized explanations, examples, and practice activities for individual learners rather than delivering the same content to every student.

---



## Spot check 2 answers

1. **What is a generative user experience?**

A generative user experience is one where content, explanations, recommendations, or workflows are dynamically created and adapted to individual users.

2. **How does an AI-native application differ from an application that simply includes AI features?**

An AI-native application is built around AI as its core capability, while a traditional application may simply add AI as an additional feature.

3. **How is generative AI changing software development workflows?**

Generative AI helps developers generate code, explain concepts, create documentation, prototype ideas, and solve problems more efficiently.

4. **What is vibe coding?**

Vibe coding is a development approach where developers use natural language instructions and AI tools to generate large portions of an application rather than writing all of the code manually.

5. **Why does code generation still require developer oversight?**

Because AI-generated code may contain bugs, security vulnerabilities, performance issues, or incorrect implementations that must be reviewed and tested.

---



## Spot check 3 answers

1. **Why do instructions play such an important role in generative AI systems?**  
Instructions guide the AI's behaviour and significantly influence the quality, relevance, and style of the generated output.
2. **How do generative systems differ from traditional software systems in the way they produce outputs?**  
Traditional software typically produces predictable outputs based on predefined logic, while generative systems create outputs dynamically based on the instructions they receive.
3. **Why might two different instructions produce different results even when using the same AI model?**  
Different instructions provide different levels of context, detail, and constraints, which influence how the model generates its response.
4. **How does prompt engineering relate to the development of generative applications?**  
Prompt engineering focuses on designing effective instructions that help AI systems generate more useful, accurate, and relevant outputs.
5. **Why is understanding instructions becoming an important skill for modern developers?**  
Because the quality of many AI-powered features depends on how effectively developers can guide and structure interactions with generative AI systems.



## Share your thoughts

Please take some time to complete this short feedback **form** to help us ensure we provide you with the best possible learning experience.

---